

Software Development in the Modern World with the Rise of Containerization

Impact of technologies such as Docker and Kubernetes on the modern world of software development

Impact of technologies such as Docker and Kubernetes on the technical writing profession

Hello.

Over the past several years, significant changes have taken place in the field of software development. For the most part, the issue of the notorious 'works on my machine' has been overcome. Technologies like Docker and Kubernetes have altered the vocabulary, processes, and documentation used by developers, thus altering approaches to creating software.

That being said, such changes present both an opportunity and a challenge for technical writers. It is necessary now to have knowledge about the new technologies in order to create relevant technical documentation for developers and DevOps specialists working with containers.

Containerization: Docker and Kubernetes

Understanding Containerization: Docker and Kubernetes

What Is Docker?

Docker is a framework created for developing containers. Docker is an open-source framework. Containers allow putting together an application with its libraries and configuration files in a single portable, self-sufficient bundle. The key feature that makes Docker superior to virtual machines is that it uses the kernel of the operating system on which the host runs.

Some of the notable Docker notions for technical writers include the following:

- Image: A read-only blueprint used to create a container.
 - Container: An image that is portable, isolated, and consistent.
-

- Docker File: A file containing instructions for creating a Docker image.
- Docker Hub: a public repository hosting Docker images.
- Image: A blueprint that is only readable and used to create a container.
- Container: a blueprint instance that is isolated, portable, and consistent.
- Docker File: A script explaining how to create a Docker image.
- Docker Hub: Repository hosting Docker images available to everyone.

What Is Kubernetes?

Kubernetes is a cloud-native container management technology created by Google and widely known as K8s. While Docker oversees the creation and execution of containers, Kubernetes supervises containers at a large scale through automated management of their deployment, scaling, networking, and recovery in clusters of servers.

Key Kubernetes notions for technical writers:

- Pod: the most basic entity of Kubernetes deployment; consists of one or several containers.
- Node: a physical or virtual machine building a Kubernetes cluster.
- Cluster: a set of nodes that utilizes Kubernetes application containers.
- Deployment: Kubernetes resource managing pod deployments.
- Service: a mechanism for defining pod sets and applying access controls to them.

Features	Docker (Build & Run)	Kubernetes (Orchestration)
Primary Units	Image: A read-only template.	Pod: A set of one or more containers.
Environment	Docker file: Script for building images.	Cluster: Set of Nodes.
Storage	Docker Hub: Online Repository.	Deployment: Manages pod rollouts.

Effect on Software Development Processes

Faster Development Cycle

Docker has enhanced the development life cycle by enabling application development and distribution to occur independently in containers. It has greatly streamlined the development of applications by eliminating delays associated with resolving issues that are usually difficult and take time. The approach has been efficient due to its ability to conduct tests and build new environments in a few seconds.

The release process has been highly optimized due to the rapid development. It is currently possible to conduct releases in just a couple of days, compared to the weeks and sometimes months it would previously take. For this reason, the documentation process needs to be optimized such that it can accommodate the changes.

Microservices Architecture

A microservices approach is now possible following the addition of Docker and Kubernetes in applications. The method entails breaking down the whole application into various units called microservices. Each unit operates individually while having its own API.

In relation to microservices documentation, each unit needs to be documented as part of the microservices approach. For this reason, technical writers will need to modify their working methods in accordance with the strategy.

Changing CI/CD Pipelines

All tools like GitLab CI, GitHub Actions, Jenkins, and any other CI/CD tools are smoothly integrated with Kubernetes. Now, the automated pipeline includes the steps of creating Docker containers, testing, and pushing the changes to the Kubernetes cluster without manual actions.

It is possible to document the pipelines, write runbooks for deployment, and define rollback strategies for those who know what a CI/CD pipeline does. In case the technology is used in containerization, everything will become particularly important.

Technical Writers' Function in Containerized Applications

Connecting the Experts to the Rest of Us

It may not be the easiest topic to grasp, but while experienced software engineers and DevOps practitioners fully comprehend the ins and outs of tools such as kubectl and YAML files, many other users, including entry-level developers, may have trouble comprehending them. This is where technical writers come into play. By familiarizing themselves with the fundamentals of technologies such as Docker and Kubernetes, they help demystify containerized applications and serve as intermediaries between the experts and everyone else.

Best Practices for Documentation of Containerized Applications

Here are some tips when documenting containerized systems:

- **Dockerfile & Compose Files:** Rather than simply laying out a list of commands in the Dockerfile, describe what the instructions mean and how one creates environment variables. Also, demonstrate how Docker Compose, through YAML configuration files, manages multiple containers in a single application.
 - **Versioned Docker Image Documentation:** Always include the versions of the Docker images in the documentation you create, or users will be at sea whenever there's an update.
 - **Operational Runbooks:** Kubernetes can be a challenge to master. Provide runbooks for common use cases, ranging from node failure to rollback of a release, scaling of deployments, and inspecting pod logs.
 - **Glossary:** There's a whole host of specialized vocabulary to be learned when dealing with containerization concepts like images, pods, namespaces, ingress controllers, and Helm Charts.
 - **Documentation as Code:** Documentation should be saved alongside your source code in Git. That makes it part of the continuous integration and delivery process.
-

Difficulties and Their Solutions

Containerized solutions constantly evolve. The release cycles for Kubernetes are always on, while other components (such as Helm, Istio, Prometheus, and Argo CD) continuously innovate, which results in out-of-date documentation being created at a faster-than-necessary pace.

These are steps that may be taken to deal with the issue:

- Documenting should be included in sprint meetings.
- Automated testing of all code examples included in the documentation should be conducted.
- One needs to participate in daily scrums and sprint reviews to learn about changes coming soon.
- Collaboration with DevOps and SRE teams is necessary.

Conclusion

The technologies related to the creation and deployment of containers have changed not only how software applications are developed but also how they are deployed. Technical authors who deal with containers no longer learn new buzzwords alone; they become specialists familiarizing themselves with various DevOps concepts, infrastructure knowledge, and culture. In today's software development companies, writing high-quality documentation about Kubernetes has become one of the requirements. With the rapid growth of containers, there will be more demand for writers capable of delivering well-written documentation.